

Hush V1 — Full UX & Interaction Specification (Watch + iPhone)

Source of truth: this document describes two working interactive prototypes built in the design conversation — a **Watch build** (compressed, Digital Crown-driven) and an **iPhone build** (full-size, the source the watch is compressed from). Both share the same 9 screens, state machine, and data model. It is written so an engineer (or Claude Code) can implement either or both without needing the original prototypes open side by side.

Target platforms: Apple Watch (watchOS) and iPhone (iOS). Watch proportions are modeled on the 45mm / Ultra display (396×484pt, aspect ratio ≈ 0.82); iPhone proportions are modeled on a standard 390×844pt display (aspect ratio ≈ 0.46). Where the prototypes used web/HTML approximations (pointer events, CSS transforms, `setInterval`), the spec calls out the native equivalent. Everything in §§1-7 applies to both builds unless explicitly flagged as device-specific; **§8 is a consolidated diff** of everything that differs between them.

0. Principle

The watch is a **compressed mirror** of the existing iPhone workout flow — not a separate product. Every screen, label, and state below has a 1:1 counterpart in the iPhone app. No new workout logic, no new terminology, no watch-native re-invention.

1. Visual system

Palette (monochrome only — no color, no semantic states, no icons beyond system glyphs):

Token	Value	Use
<code>bg.screen</code>	 <code>#000000</code>	Screen background, always
<code>bg.sheet</code>	 <code>#161616</code>	Bottom sheets (Pause, Edit Result)
<code>bg.case</code>	 <code>#131315</code>	Watch case / bezel (chrome, not screen content)
<code>text.primary</code>	 <code>#FFFFFF</code>	Primary numbers, titles, primary buttons
<code>text.secondary</code>	<code>rgba(255,255,255,0.45)</code>	Labels, captions, set counters
<code>text.tertiary / dim</code>	<code>rgba(255,255,255,0.25-0.28)</code>	Carried-over context behind sheets (e.g. dimmed weight under Pause/Edit)
<code>fill.subtle</code>	<code>rgba(255,255,255,0.12)</code>	Slide track, picker highlight pill, step buttons

Typography: system font (SF Compact on-device; `-apple-system` fallback elsewhere). Two functional weights: **regular** (secondary text) and **bold** (everything that is a primary value or action — this app's numbers are meant to read as heavy, not default-weight).

Element	Watch	iPhone	Weight
Hero number (weight on Active Set, "Well done")	42pt	72pt	bold
Countdown timer	38pt, tabular	62pt, tabular	bold
Wheel picker — selected row	17pt	24pt	bold
Exercise / sheet title	14-16pt	18-20pt	bold
Body / secondary (reps, load, eta)	11-13pt	15-16pt	regular
Coaching line	12pt	15pt	regular

iPhone type isn't a uniform multiple of the Watch scale (it's not "Watch × 1.7" everywhere) — each size was set to feel native at arm's length on the bigger screen, not mechanically derived.

Geometry — Watch: case corner radius ≈ 24% of case width (squircle); screen inset 12pt from case edge; screen corner radius ≈ case radius – inset.

Geometry — iPhone: thinner bezel inset (≈10pt, proportionally smaller on the larger frame); Dynamic Island rendered as a near-black pill at top center (it visually disappears against the pure-black screen content — that's correct, expected behavior, not a rendering

gap); a persistent home-indicator bar sits at the bottom edge on every screen, outside any individual screen's own layout.

No drop shadows, no gradients, no glow on either build — flat fills only.

No color, no emoji, no decorative icons anywhere, including on destructive actions (End workout). Hierarchy is conveyed entirely through weight, size, and opacity.

2. Data model

```
Workout {
  name: string          // "Upper A"
  muscles: string        // "Chest · Shoulders · Triceps"
  exercises: Exercise[]
}

Exercise {
  name: string           // "Bench press"
  lastWeek: number       // reference weight for coaching copy
  sets: Set[]
}

Set {
  weight: number         // kg, 2.5 increments (placeholder – should
                        // come from the exercise's real equipment
                        // increment:
  barbell/dumbbell/machine/plate)
  reps: number
}
```

Session state (lives in the watch app's view model, not per-screen):

```

currentWorkout: string
exerciseIndex: number
setIndex: number
etaSecondsRemaining: number    // ticks down once per second while active
restSecondsRemaining: number    // ticks down only on rest screens
isPaused: boolean
prePauseScreen: ScreenID        // where to return on "Resume workout"
editDraft: { weight, reps }     // scratch values while the picker is open
activeWheel: "weight" | "reps"  // Watch only. Which picker column the Crown
                                // currently drives. Not needed on iPhone —
                                // both wheel columns are independently
                                // touch-draggable at once, so there's no
                                // single "focused" column to track.

```

3. Screens

3.1 Start

Purpose: mirrors the iPhone home workout card. **Content:** date · greeting (top-left) · "Choose workout" pill (top-right, replaces the hamburger) · workout name (large, bold) · muscle groups (secondary) · "Slide to start" control pinned to the bottom edge.

Interactions:

- Tap "Choose workout" → opens a full-screen list overlay (Upper A / Upper B / Lower A / Lower B / ...). Tapping an item updates the workout shown on this screen immediately and closes the list. No confirmation step.
- "Slide to start" — drag the thumb right; release past ~70% of track width commits. A direct tap also commits (treat as "intent confirmed without friction" — see §7 for the safety trade-off to confirm with design). **Transition out:** commit → **Active Set** (exercise 0, set 0). Reset `etaSecondsRemaining` from the workout's total estimate.

3.2 Active Set

Purpose: the watch adaptation of the iPhone Active Set screen. **Content:** header — eta remaining (top-left) + pause glyph (top-right) · exercise name · hero weight number with "kg" suffix · reps (`(× 8)`) · "Set X of Y" · coaching line · footer — "Edit result" (text link) + "Complete set" (primary pill button). **Coaching line logic:**

```

diff = currentSet.weight - exercise.lastWeek
if diff > 0:  "Up {diff} from last week. You're ready for it."
else:        "You can do it."

```

Interactions: tap pause glyph → **Pause Sheet** (preserves return screen). Tap "Edit result" → **Edit Result**. Tap "Complete set" → **Set Confirmation**.

3.3 Set Confirmation

Purpose: brief, non-interactive acknowledgement, identical in spirit to the iPhone's post-log state. **Content:** `{weight} × {reps}` only. Same header (eta) carried over, no pause control needed (screen is too short-lived to need it). **Behavior:** fully automatic. After **1.1s**, advance:

```
if there is a next set in this exercise:      → Inter-Set Rest (next set's
data)
else if there is a next exercise in workout:  → Transition Rest (next
exercise's first set)
else:                                         → Well Done
```

Note: the indices (`setIndex`)/(`exerciseIndex`) advance **before** rendering the rest screen, so the rest screen always previews the *upcoming* set/exercise, not the one just completed.

3.4 Inter-Set Rest

Purpose: rest between sets of the same exercise. **Content:** header (eta + pause) · large countdown timer · "Set X of Y" · exercise name · upcoming weight × reps · "Ready" control at the bottom, plus one device-specific status line directly below "Ready" (full detail in §8):

Watch shows heart rate + calories, e.g. `118 bpm 142 kcal`; **iPhone** shows the literal text

`Exercise Busy`. Never both on the same build, never icons or color either way. **Behavior:**

`restSecondsRemaining` ticks down once per second (paused if `isPaused`). Reaching 0 auto-advances to **Active Set**. Tapping "Ready" advances immediately (skips remaining rest).

Tapping the pause glyph → **Pause Sheet**.

3.5 Transition Rest

Purpose: rest before the next exercise. Structurally identical to Inter-Set Rest, with one content difference: label reads "Next exercise" instead of "Set X of Y", and shows the next exercise's name + its first set's load/reps. Same countdown/auto-advance/"Ready"/pause behavior as §3.4, and the same device-specific status line below "Ready" (bpm/kcal on Watch, `Exercise Busy` on iPhone — see §8).

3.6 Edit Result

Purpose: adjust the weight/reps just logged (or about to be logged), reachable from Active Set. **Content:** dimmed carried-over weight number at the top (continuity cue, ~26% opacity) · a sheet with two independent columns, **Weight** and **Reps**, each showing 5 rows (2 above, selected, 2 below) with the selected row highlighted in a pill · a Save button. **Watch** — **Digital Crown model:** only one column is "active" (driven by the Crown) at a time.

- Tapping a column makes it active: active column → full opacity/scale; inactive column → dimmed (~40% opacity) and slightly scaled down (0.94×). This is the **required** feedback mechanism — the user must always be able to tell which value the Crown currently controls.

- Rotating the Crown while a column is active changes that column's value by one increment per detent (weight: $\pm 2.5\text{kg}$, reps: ± 1). On-device this is `.digitalCrownRotation` bound to the active column's value; the prototype's mouse-wheel listener and $\pm$ stepper buttons are stand-ins for this.
- A short haptic tick on each increment, and on switching the active column, is expected on-device (not simulated in the web prototype).

iPhone — direct touch model: no "active column" concept; both wheels are simultaneously live, since touch (unlike a single Crown) can address either column directly.

- Each column is its own vertical drag surface: dragging up/down changes that column's value by one increment per $\sim 30\text{pt}$ of travel (weight: $\pm 2.5\text{kg}$, reps: ± 1); release commits wherever it lands, no separate confirm step.
- Both columns can be touched independently and at the same time; neither dims the other — there's nothing to disambiguate.
- Arrow-key ($\uparrow/\downarrow$) support is included for accessibility/keyboard use, applying the same increment.
- A light haptic tick per increment is expected on-device, same intent as the Watch version, just not tied to a Crown detent.

Interactions (both): tap "Save" $\rightarrow$ commits `editDraft` into the current set and returns to **Active Set**, which re-renders with the new weight/reps (and re-evaluates the coaching line).

3.7 Pause Sheet

Purpose: pause the active workout. **Content:** dimmed carried-over weight at top (same continuity treatment as Edit Result) · sheet titled "Workout paused". **Watch:** two items — "Resume workout" (primary), "Finish early" (secondary); "Show exercise" is intentionally omitted to keep the sheet reachable without the Crown. **iPhone:** three items — "Resume workout" (primary), "Show exercise", "Finish early" — matching the source app exactly, since there's room for it and no input-method reason to cut it. **Interactions:** "Resume workout" $\rightarrow$ returns to whichever screen was active when pause was triggered (`prePauseScreen`), with all timers un-paused. "Finish early" $\rightarrow$ **End Workout Confirmation**. "Show exercise" (iPhone only) — present in the prototype as a static row; its actual behavior is intentionally left undefined here. Flag with design whether it should surface exercise media/instructions or simply return to the active screen. **Side effect of entering this screen:** both the eta timer and any running rest countdown stop ticking; they resume exactly where they left off on "Resume workout".

3.8 End Workout Confirmation

Purpose: a deliberate second step before destructive action — prevents an accidental tap on "Finish early" from ending the session. **Content:** "End workout?" · "Resume workout" (primary) · "End workout" (text action, bold — kept monochrome per the no-color rule, not red, since none of the reference screens use color for destructive actions; flag this with

design if a red accent is actually wanted here). **Interactions:** "Resume workout" → same return-to-(prePauseScreen) behavior as §3.7. "End workout" → **Well Done** (workout is terminated immediately, no further confirmation).

3.9 Well Done

Purpose: session-complete acknowledgement, identical to the iPhone's end screen.

Content: "Well done." — large, bold, no buttons. (Reference design includes a small square accent dot after the text, not a literal period — purely typographic, not a status indicator.)

Behavior: fully automatic, no input accepted. Auto-dismisses after **2-3 seconds** (prototype uses 2.6s) and returns to **Start**, with session state reset (`exerciseIndex = 0`, `setIndex = 0`, eta recalculated for whatever workout is selected next).

4. Global behaviors

- **ETA countdown:** ticks down once per second whenever the current screen is Active Set, Set Confirmation, Inter-Set Rest, or Transition Rest, and `isPaused` is false. Displayed wherever the header is present, formatted as `{minutes}m left`. Starting value should be sourced from the same workout-duration estimate the iPhone app already computes (the prototype uses a placeholder: `totalSets × 200s`).
 - **Rest countdown:** only ticks on the two rest screens; independent of the eta timer; resets to a fresh duration each time a rest screen is entered (placeholder defaults: 90s inter-set, 120s transition — production should pull the real prescribed rest period per exercise).
 - **Pause is global:** triggered from the pause glyph on any in-workout screen. It freezes both timers and shows the Pause Sheet; "Resume workout" restores the exact prior screen and unfreezes both timers.
 - **No screen in this flow displays color, an icon set beyond pause/chevron/check, or any "busy/status" badge.** Hierarchy is weight/size/opacity only.
-

5. State transition table

From	Trigger	To
Start	Slide to start committed	Active Set (ex 0, set 0)
Start	Tap "Choose workout", select item	Start (workout swapped, no nav)
Active Set	Tap "Complete set"	Set Confirmation
Active Set	Tap "Edit result"	Edit Result

From	Trigger	To
Active Set	Tap pause glyph	Pause Sheet
Set Confirmation	1.1s elapsed, more sets remain	Inter-Set Rest
Set Confirmation	1.1s elapsed, exercise done, more exercises remain	Transition Rest
Set Confirmation	1.1s elapsed, workout done	Well Done
Inter-Set Rest	Timer hits 0, or "Ready" tapped	Active Set
Inter-Set Rest	Tap pause glyph	Pause Sheet
Transition Rest	Timer hits 0, or "Ready" tapped	Active Set
Transition Rest	Tap pause glyph	Pause Sheet
Edit Result	Tap "Save"	Active Set (values updated)
Pause Sheet	Tap "Resume workout"	<code>prePauseScreen</code>
Pause Sheet	Tap "Finish early"	End Workout Confirmation
End Workout Confirmation	Tap "Resume workout"	<code>prePauseScreen</code>
End Workout Confirmation	Tap "End workout"	Well Done
Well Done	2-3s elapsed	Start (state reset)

6. Component specs

Slide to Start — pill track, 42pt tall, full width, background `fill.subtle`; circular white thumb (36pt) with a trailing chevron glyph; label centered in the track, behind the thumb, non-interactive. Drag the thumb (or tap the track) to commit.

Choose Workout overlay — full-screen list, near-opaque black background, simple text rows with hairline dividers, no search/filter in V1.

Wheel picker (Weight / Reps) — two columns, 5 visible rows each, center row in a rounded-pill highlight, independent scroll state per column. The two builds drive it differently: on **Watch**, only the active column responds to the Crown and the inactive one dims, because a single physical Crown needs an explicit "which value am I touching" affordance. On **iPhone**, both columns are independently touch-draggable at once with no dimming, since touch can address either column directly without ambiguity. Full detail in §3.6 and §8.

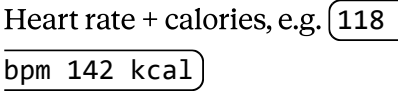
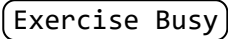
Bottom sheets (Pause, Edit Result) — slide up from the bottom edge, rounded top corners only, `bg.sheet`, cover roughly the lower half to two-thirds of the screen, dimmed content visible behind for continuity.

7. Open items to confirm before build

1. **Slide-to-start "tap also commits"** — the prototype allows a plain tap to count as a full slide, for demo convenience. Production should probably require an actual drag gesture (or a long-press) to avoid accidental workout starts from a stray wrist tap. Recommend confirming the threshold/gesture with design before shipping.
 2. **"End workout" styling** — kept monochrome here to match the reference screens exactly, but most watchOS conventions use a red destructive action. Worth a deliberate yes/no rather than defaulting either way.
 3. **Rest durations and ETA formula** — both are placeholders in the prototype; should be wired to whatever the iPhone app already uses so the numbers agree across devices.
 4. **Weight increments** — 2.5kg used throughout; should vary by equipment type (barbell vs. dumbbell vs. machine) per the real exercise library.
 5. **Crown haptics** — expected on real hardware for both value increments and active-column switches; not something a non-watchOS prototype can demonstrate, called out here so it isn't missed during implementation.
-

8. Device differences: Watch vs iPhone — consolidated

Everything not listed here is identical between the two builds: same 9 states, same transitions (§5), same data model (§2), same copy, same monochrome visual system, same global timer behavior (§4).

Aspect	Watch	iPhone
Frame	~250×305pt case · 0.82 aspect ratio · Digital Crown + side button	~300×650pt case · 0.46 aspect ratio · Dynamic Island + home indicator
Type scale	Compact — 42pt hero / 38pt timer	Large — 72pt hero / 62pt timer (full table in §1)
Edit Result interaction	One "active" column at a time, Crown-driven, explicit focus state required	Both columns independently touch-draggable at once, no focus state needed
Inter-Set / Transition Rest status line	Heart rate + calories, e.g. 	Literal text 
Pause Sheet items	Resume workout, Finish early (2 items)	Resume workout, Show exercise, Finish early (3 items)
Why they differ	Compressed for a glanceable, Crown/wrist-driven surface	Full fidelity to the original source screens — no input-method constraint to compress against

None of these differences touch the state machine (§5) or the data model (§2) — a single shared view-model can drive both UIs. Only the view layer and the two flagged interaction points (Edit Result input method, Pause Sheet item count) branch by device; everything else is the same component tree at a different scale.